



Explain child to parent communication in LWC in detail with multiple examples and explain all the examples

In **Lightning Web Components (LWC)**, communication between a **child component to a parent component** can be achieved using **Custom Events**. This is a standard pattern where the child component dispatches an event, and the parent listens for it.

Child to Parent Communication in LWC

✓ Basic Steps:

1. **Child Component:** Dispatches a custom event using `this.dispatchEvent(new CustomEvent('eventname', { detail }))`
2. **Parent Component:** Listens to the event in its template and handles it in a JS method.

Example 1: Sending simple data (like a string)

 **Goal:** Child sends a name string to the parent.

 **Child Component:** `childComponent.js`

```
js

import { LightningElement } from 'lwc';

export default class ChildComponent extends LightningElement {
  sendName() {
    const name = "John Doe";
    this.dispatchEvent(new CustomEvent('sendname', {
      detail: name
    }));
  }
}
```

`childComponent.html`

```
html

<template>
  <lightning-button label="Send Name" onclick={sendName}></lightning-button>
</template>
```

 **Parent Component:** `parentComponent.js`

```
js

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  receivedName;

  handleName(event) {
    this.receivedName = event.detail;
  }
}
```

parentComponent.html

```
html

<template>
  <c-child-component onsendname={handleName}></c-child-component>
  <p>Received Name: {receivedName}</p>
</template>
```

Example 2: Sending an object from child to parent

 **Goal:** Child sends a contact object (name and email)

 **Child Component:** childComponent.js

```
js

import { LightningElement } from 'lwc';

export default class ChildComponent extends LightningElement {
  sendContact() {
    const contact = {
      name: 'Alice',
      email: 'alice@example.com'
    };
    this.dispatchEvent(new CustomEvent('contactinfo', {
      detail: contact
    }));
  }
}
```

childComponent.html

```
html

<template>
  <lightning-button label="Send Contact Info" onclick={sendContact}></lightning-button>
</template>
```

 **Parent Component:** parentComponent.js

```
js

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  contact;

  handleContactInfo(event) {
    this.contact = event.detail;
  }
}
```

parentComponent.html

```
html

<template>
  <c-child-component oncontactinfo={handleContactInfo}></c-child-component>
  <p>Name: {contact.name}</p>
  <p>Email: {contact.email}</p>
</template>
```

Example 3: Event with bubbles and composed properties

 **Goal:** Make event go up multiple DOM levels (if needed)

```
js

this.dispatchEvent(new CustomEvent('someevent', {
  detail: { ... },
  bubbles: true,
  composed: true
}));
```

- **bubbles: true:** lets the event bubble up the DOM tree
- **composed: true:** lets it cross shadow DOM boundaries

 Use when your parent is not the immediate container or using SLDS/modal frameworks.

Example 4: Using parameters dynamically

 **Goal:** Child sends a selected value from dropdown to parent

 **Child:** childComponent.html

```
html

<template>
  <lightning-combobox
    name="options"
    label="Choose Option"
    value={value}
    options={options}
    onchange={handleChange}>
```

```
</lightning-combobox>
</template>
```

childComponent.js

```
js

import { LightningElement } from 'lwc';

export default class ChildComponent extends LightningElement {
  value = '';
  options = [
    { label: 'One', value: '1' },
    { label: 'Two', value: '2' }
  ];

  handleChange(event) {
    this.value = event.detail.value;
    this.dispatchEvent(new CustomEvent('optionselected', {
      detail: this.value
    }));
  }
}
```

Parent: parentComponent.html

```
html

<template>
  <c-child-component onoptionselected={handleOption}></c-child-component>
  <p>You selected: {selectedOption}</p>
</template>
```

parentComponent.js

```
js

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  selectedOption;

  handleOption(event) {
    this.selectedOption = event.detail;
  }
}
```

Summary

Feature	How it Works
<code>dispatchEvent()</code>	Used in child to emit an event
<code>CustomEvent('name')</code>	Defines the event name (listened by parent)
<code>detail: {...}</code>	Carries the payload to the parent
<code>onsomeevent={handler}</code>	Listener in parent HTML

Feature	How it Works
bubbles & composed	Allow deeper/shadow DOM event propagation if needed

Pro Tips

- Always **document** your custom event names to avoid confusion.
- Use constants for event names if reused.
- Avoid naming conflicts between native events and custom events (**click**, **change** etc.).
- You can dispatch events in child-to-child via intermediary parent using this model.

Would you like an advanced example involving multiple child components communicating via the parent?



Now for parent to child

Great—let's tackle **Parent to Child communication in Lightning Web Components (LWC)** in detail.

Parent to Child Communication in LWC

Unlike child-to-parent, **Parent to Child** is usually much simpler. You don't need custom events; instead, you do this primarily via:

- ✓ **Public Properties (@api decorated variables)**
- ✓ **Public Methods (@api decorated functions)**
- ✓ **Slots** (for content injection—like transclusion)
- ✓ **Template Queries (querySelector)**—not recommended for production usually

I'll walk you step by step through each with examples.

Example 1: Passing data from parent to child via public properties

 **Goal: Parent provides a greeting string to the child.**

 **Child Component:** `childComponent.js`

```
javascript

import { LightningElement, api } from 'lwc';

export default class ChildComponent extends LightningElement {
```

```
@api greeting; // public property
}
```

childComponent.html

```
html

<template>
  <p>{greeting}</p>
</template>
```

👤👤 Parent Component: parentComponent.html

```
html

<template>
  <c-child-component greeting="Hello from Parent!"></c-child-component>
</template>
```

✅ Explanation:

- The `@api` decorator makes the `greeting` property accessible from the parent.
- The parent sets `greeting="..."` directly in markup.

🟢 Example 2: Dynamically binding data to child

🎯 Goal: Parent has a property, passed dynamically to child.

👤👤 Parent Component: parentComponent.js

```
javascript

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  parentMessage = 'Dynamic Message';
}
```

parentComponent.html

```
html

<template>
  <c-child-component greeting={parentMessage}></c-child-component>
</template>
```

✅ Explanation:

- `{parentMessage}` is dynamic.
- If `parentMessage` changes, the child automatically re-renders.

● Example 3: Invoking child component methods via `@api`

This is more interactive: **Parent calls a function on the child.**

 **Goal:** Parent triggers a method on the child component (e.g., reset form).

 **Child Component:** `childComponent.js`

```
javascript

import { LightningElement, api } from 'lwc';

export default class ChildComponent extends LightningElement {
  message = 'Initial Message';

  @api
  resetMessage() {
    this.message = 'Reset by Parent!';
  }
}
```

childComponent.html

```
html

<template>
  <p>{message}</p>
</template>
```

 **Parent Component:** `parentComponent.js`

```
javascript

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  handleReset() {
    this.template.querySelector('c-child-component').resetMessage();
  }
}
```

parentComponent.html

```
html

<template>
  <lightning-button label="Reset Child Message" onclick={handleReset}></lightning-button>
```

```
<c-child-component></c-child-component>
</template>
```

✓ Explanation:

- `@api resetMessage()` exposes the method to the parent.
- `this.template.querySelector('c-child-component')` gets the child instance.
- `resetMessage()` is invoked when the parent button is clicked.

● Example 4: Using slots (Parent injects markup into child)

This is useful for custom layout/content.

🎯 Goal: Parent injects custom markup into the child slot.

👤 Child Component: `childComponent.html`

```
html

<template>
  <div class="slot-container">
    <slot></slot>
  </div>
</template>
```

✓ Note: No JS needed here.

👤👤 Parent Component: `parentComponent.html`

```
html

<template>
  <c-child-component>
    <p>This content comes from Parent!</p>
    <lightning-button label="A button inside slot"></lightning-button>
  </c-child-component>
</template>
```

✓ Explanation:

- The `<slot>` acts as a placeholder.
- Anything between `<c-child-component>...</c-child-component>` shows inside the child.

● Example 5: Combining property passing and method invocation

 **Goal: Parent sets an initial value, then later resets via method.**

Child Component: childComponent.js

```
javascript

import { LightningElement, api } from 'lwc';

export default class ChildComponent extends LightningElement {
  @api greeting;

  @api
  resetGreeting() {
    this.greeting = 'Reset by Parent!';
  }
}
```

childComponent.html

```
html

<template>
  <p>{greeting}</p>
</template>
```

Parent Component: parentComponent.js

```
javascript

import { LightningElement } from 'lwc';

export default class ParentComponent extends LightningElement {
  message = 'Hello initially';

  handleReset() {
    this.template.querySelector('c-child-component').resetGreeting();
  }
}
```

parentComponent.html

```
html

<template>
  <lightning-button label="Reset Greeting" onclick={handleReset}></lightning-button>
  <c-child-component greeting={message}></c-child-component>
</template>
```

Example 6: Accessing multiple child components

If you have multiple children, you can:

- ✓ Use `querySelectorAll()`
- ✓ Call their methods individually

```
javascript

handleResetAll() {
  const children = this.template.querySelectorAll('c-child-component');
  children.forEach(child => {
    child.resetGreeting();
  });
}
```

Quick Recap of Parent to Child Communication Techniques

Technique	How It Works
Public Properties (<code>@api</code>)	Parent sets attribute/property on the child in markup.
Public Methods (<code>@api</code>)	Parent calls child method using <code>querySelector()</code> .
Slots	Parent injects markup content into child component slots.
Query Selectors	Used to locate child components in parent template (use carefully).

✓ Best Practice:

- Use **public properties and methods** for clear API.
- Avoid `querySelector()` except for calling `@api` methods—don't rely on it for accessing internals.
- Keep child components loosely coupled (child shouldn't rely on parent specifics).

If you'd like, I can also show you advanced scenarios, like:

- **Communicating across non-direct relationships** (siblings)
- **Using Lightning Message Service**

Just let me know!



Explain navigation in Salesforce in detail with multiple examples and explain all the examples

Absolutely—let's walk step by step through **navigation in Salesforce Lightning Web Components (LWC)** in detail.

This is a big topic, so I'll organize it carefully:

Navigation in Salesforce (Lightning Web Components)

Navigation means programmatically telling Salesforce to take the user to:

- ✓ A record page
- ✓ A list view
- ✓ A web URL
- ✓ A related list
- ✓ A Lightning Component
- ✓ A Visualforce page
- ✓ A file preview

In LWC, you use the **NavigationMixin** from `lightning/navigation`.

How Navigation Works

1. Import **NavigationMixin**
2. Extend your component with **NavigationMixin**
3. Call `this[NavigationMixin.Navigate](pageReference)`

✓ Syntax Recap

```
javascript

import { NavigationMixin } from 'lightning/navigation';

export default class MyComponent extends NavigationMixin(LightningElement) {
  myNavigationMethod() {
    this[NavigationMixin.Navigate]({
      type: 'standard__objectPage' | 'standard__recordPage' | 'standard__navItemPage' |
      ...,
      attributes: { /* destination-specific attributes */ },
      state: { /* optional query params */ }
    });
  }
}
```

I'll show you examples of all common types.

Example 1: Navigate to a Record Page (View Mode)

 **Goal:** Navigate to a specific record in view mode.

 **myComponent.js**

```
javascript

import { LightningElement } from 'lwc';
import { NavigationMixin } from 'lightning/navigation';

export default class MyComponent extends NavigationMixin(LightningElement) {
  navigateToRecordView() {
    this[NavigationMixin.Navigate]({
      type: 'standard__recordPage',

```

```

        attributes: {
            recordId: '001XXXXXXXXXXXXXXX', // Replace with your record Id
            objectApiName: 'Account',
            actionName: 'view'
        }
    });
}
}
}

```

myComponent.html

```

html

<template>
    <lightning-button label="Go to Record View" onclick={navigateToRecordView}></lightning-
button>
</template>

```

✓ Explanation:

- **type:** 'standard__recordPage' means you're opening a record.
- **actionName:** 'view' shows the record.
- When the button is clicked, user is navigated to the record's detail page.

● Example 2: Navigate to Record Page (Edit Mode)

 **Goal:** Open the same record in edit mode.

Just change **actionName:** 'edit'.

```

javascript

navigateToRecordEdit() {
    this[NavigationMixin.Navigate]({
        type: 'standard__recordPage',
        attributes: {
            recordId: '001XXXXXXXXXXXXXXX',
            objectApiName: 'Account',
            actionName: 'edit'
        }
    });
}
}

```

✓ Usage:

- Use this when you want users to jump straight into editing.

● Example 3: Navigate to New Record Page

 **Goal:** Open the New Record creation page for an object.

```
javascript

navigateToNewRecord() {
  this[NavigationMixin.Navigate]({
    type: 'standard__objectPage',
    attributes: {
      objectApiName: 'Opportunity',
      actionName: 'new'
    }
  });
}
```

✓ Explanation:

- **type:** 'standard__objectPage'
- **actionName:** 'new' opens the "New" record form.

● Example 4: Navigate to List View

 **Goal:** Take the user to the list view of an object.

```
javascript

navigateToListView() {
  this[NavigationMixin.Navigate]({
    type: 'standard__objectPage',
    attributes: {
      objectApiName: 'Contact',
      actionName: 'list'
    },
    state: {
      filterName: 'Recent' // or a specific list view Id
    }
  });
}
```

✓ Notes:

- **filterName:** 'Recent' shows the "Recently Viewed" list.
- You can also pass your list view API name.

● Example 5: Navigate to Related List

 **Goal:** Open the related list of child records.

```
javascript

navigateToRelatedList() {
  this[NavigationMixin.Navigate]({
    type: 'standard__recordRelationshipPage',
    attributes: {
      recordId: '001XXXXXXXXXXXXXXXXX',
      objectApiName: 'Account',

```

```

        relationshipApiName: 'Contacts',
        actionName: 'view'
    });
}

```

✓ Explanation:

- **recordId** is the parent record.
- **relationshipApiName** is the child relationship (e.g., Contacts).
- This shows the related list.

● Example 6: Navigate to a Custom Tab (App Navigation Item)

 **Goal:** Navigate to a custom Lightning App Tab (Lightning Page).

```

javascript

navigateToCustomTab() {
    this[NavigationMixin.Navigate]({
        type: 'standard__navItemPage',
        attributes: {
            apiName: 'My_Custom_Tab'
        }
    });
}

```

✓ How to get apiName:

- Setup > Tabs > find your tab name.

● Example 7: Navigate to a Web URL

 **Goal:** Open an external website.

```

javascript

navigateToWebPage() {
    this[NavigationMixin.Navigate]({
        type: 'standard__webPage',
        attributes: {
            url: 'https://www.salesforce.com'
        }
    });
}

```

✓ **Tip:** Use HTTPS URLs to avoid mixed-content warnings.

● Example 8: Navigate to Visualforce Page

 **Goal:** Navigate to a Visualforce page in your org.

```
javascript

navigateToVFPage() {
  this[NavigationMixin.Navigate]({
    type: 'standard__webPage',
    attributes: {
      url: '/apex/MyVisualforcePage'
    }
  });
}
```

✓ Note:

- This approach requires your Visualforce page to be exposed to the user.
- The `/apex/` prefix is standard.

● Example 9: Navigate to File Preview

 **Goal:** Open a file preview of a Content Document.

```
javascript

navigateToFilePreview() {
  this[NavigationMixin.Navigate]({
    type: 'standard__recordPage',
    attributes: {
      recordId: '069XXXXXXXXXXXXXXX', // ContentDocument Id
      objectApiName: 'ContentDocument',
      actionName: 'view'
    }
  });
}
```

✓ Tip:

- You can also use `standard__namedPage` for Chatter file previews.

● Example 10: Navigate to Named Page (e.g., Today's Tasks)

 **Goal:** Use standard Salesforce named pages.

```
javascript

navigateToToday() {
  this[NavigationMixin.Navigate]({
    type: 'standard__namedPage',
```

```

attributes: {
  pageName: 'today'
}
});
}

```

✓ Available pageNames include:

- home
- today
- chatter
- work
- etc.

⚠ Important Considerations

- ✓ You **must** run **NavigationMixin** in the context of Salesforce UI (not inside an iframe that doesn't have access).
- ✓ You **can't** navigate to certain Lightning Experience pages if the user doesn't have access.
- ✓ You **can** pass URL parameters in **state**.

Example:

```

javascript

state: {
  filterName: 'AllOpportunities',
  c__customParam: 'myValue'
}

```

- c__ prefix is required for custom parameters.

🧭 Quick Reference Table

Destination	type	Key Attributes
Record Page	standard__recordPage	recordId, objectApiName, actionName
Object List View	standard__objectPage	objectApiName, actionName (list), state for filters
New Record Page	standard__objectPage	objectApiName, actionName: 'new'
Related List	standard__recordRelationshipPage	recordId, objectApiName, relationshipApiName, actionName
Custom Tab	standard__navItemPage	apiName
Web Page	standard__webPage	url
Visualforce Page	standard__webPage	url

Destination	type	Key Attributes
Named Page	<code>standard__namedPage</code>	<code>pageName</code>
File Preview	<code>standard__recordPage</code>	<code>recordId</code> , <code>objectApiName</code> : <code>'ContentDocument'</code> , <code>actionName</code>

✓ Best Practice:

- Always test navigation in Lightning Experience and Console Apps.
- Use `NavigationMixin.GenerateUrl()` if you only want the URL and don't want to navigate yet.

If you'd like, I can also show you **dynamic URL generation**, or **using Navigation in Aura Components**, or **passing custom state parameters**.

Just let me know!



Js properties, component lifecycle and composition

Absolutely—let's break these 3 core topics of **Lightning Web Components (LWC)** down carefully and in detail:

1 JavaScript Properties in LWC

JavaScript properties are the way components store and manage data.

They can be:

- ✓ Public (exposed to parent components)
- ✓ Private (internal to the component)

📌 Public Properties (@api)

- Decorated with `@api`
- Accessible from parent components
- Useful for data input, configuration, and child-to-parent communication

✓ Example:

```
javascript

import { LightningElement, api } from 'lwc';

export default class ChildComponent extends LightningElement {
  @api recordId;
  @api label = 'Default Label';

  @api
  updateLabel(newLabel) {
    this.label = newLabel;
  }
}
```

```
}
}
```

✓ Usage in parent:

```
html

<c-child-component record-id="001XXXXXXXXXXXXX" label="My Label"></c-child-component>
```

- `recordId` and `label` are set by parent.
- `updateLabel()` can be called by parent via JavaScript (if needed).

Private Properties

- **NOT** decorated with `@api`
- Only used inside the component
- For internal state or logic

✓ Example:

```
javascript

import { LightningElement } from 'lwc';

export default class MyComponent extends LightningElement {
  greeting = 'Hello';
}
```

Reactive Properties

All properties in LWC are **reactive**:

- When you assign a new value (`this.greeting = 'Hi'`), the template automatically re-renders.

✓ Example:

```
javascript

greeting = 'Hello';

handleChange(event) {
  this.greeting = event.target.value;
}
```

This will re-render the `<template>` if it uses `{greeting}`.

Tracked Properties (@track)

In modern LWC, you rarely need `@track`.

- Primitive properties (strings, numbers) are reactive by default.
- For **objects and arrays**, reassign the whole reference to trigger reactivity.

✓ Example without `@track`:

```
javascript

myObj = { name: 'Test' };

updateName() {
  // This does NOT trigger re-render:
  // this.myObj.name = 'Updated';

  // This DOES trigger re-render:
  this.myObj = { ...this.myObj, name: 'Updated' };
}
```

Note: `@track` used to be needed for nested reactivity, but no longer required if you reassign the object.

2 Component Lifecycle

LWC components go through a predictable lifecycle:

```
scss

constructor()
  ↓
connectedCallback()
  ↓
renderedCallback()
  ↓
disconnectedCallback()
```

✓ Detailed Explanation:

◆ `constructor()`

- Called when component instance is created
- DO NOT access the DOM or wire services here
- Initialize internal state only

✓ Example:

```
javascript

constructor() {
  super();
  console.log('Constructor called');
}
```

◆ `connectedCallback()`

- Called when component is inserted into the DOM

- Good for:
 - Initializing data
 - Calling Apex
 - Setting up event listeners

✓ Example:

```
javascript

connectedCallback() {
  console.log('Connected to DOM');
}
```

◆ renderedCallback()

- Called after every render of the component
- Can be called multiple times
- Be careful to avoid infinite loops
- Good for interacting with the rendered DOM

✓ Example:

```
javascript

renderedCallback() {
  console.log('Rendered or re-rendered');
}
```

✓ Common pattern:

```
javascript

renderedCallback() {
  if (this.hasRendered) return;
  this.hasRendered = true;
  // Do one-time DOM logic here
}
```

◆ disconnectedCallback()

- Called when component is removed from DOM
- Good for cleanup (e.g., remove event listeners, cancel timeouts)

✓ Example:

```
javascript

disconnectedCallback() {
  console.log('Component removed from DOM');
}
```

✓ Lifecycle Sequence Recap:

1. `constructor()`
2. `connectedCallback()`
3. `renderedCallback()` (one or more times)
4. `disconnectedCallback()`

Component Composition

Composition means putting components inside each other.

✓ **Parent component** uses child components in its template.

✓ **Benefits:**

- Reusability
- Modularity
- Separation of concerns

Example: Basic Composition

childComponent.js

```
javascript

import { LightningElement, api } from 'lwc';
export default class ChildComponent extends LightningElement {
  @api message;
}
```

childComponent.html

```
html

<p>{message}</p>
```

parentComponent.html

```
html

<template>
  <c-child-component message="Hello from parent"></c-child-component>
</template>
```

✓ Explanation:

- Parent passes `message` property.
- Child displays it.

Example: Composition with Event Communication

✓ Child emits event:

childComponent.js

```
javascript

handleClick() {
  this.dispatchEvent(new CustomEvent('mycustomclick', {
    detail: 'Button clicked'
  }));
}
```

childComponent.html

```
html

<lightning-button label="Click Me" onclick={handleClick}></lightning-button>
```

✓ Parent listens:

parentComponent.html

```
html

<c-child-component onmycustomclick={handleChildEvent}></c-child-component>
```

parentComponent.js

```
javascript

handleChildEvent(event) {
  console.log('Child says:', event.detail);
}
```

✓ Explanation:

- Child emits `mycustomclick` event.
- Parent handles it with `handleChildEvent()`.

Composition Best Practices

- Use public properties (`@api`) for data input.
- Use custom events for data output.
- Keep components **focused** on single responsibilities.

Recap Cheat Sheet

✓ JavaScript Properties

- `@api` = public, exposed
- Private = internal
- Reactive by default
- Reassign objects to trigger re-render

✓ Lifecycle

- `constructor()` – setup state
- `connectedCallback()` – initialization logic
- `renderedCallback()` – DOM handling
- `disconnectedCallback()` – cleanup

✓ Composition

- Parent uses `<c-child-component>`
- Parent -> Child = `@api` props
- Child -> Parent = Custom Events

✓ If you'd like, I can prepare **more examples** or help you design components for specific use cases (e.g., dynamic forms, data tables, navigation). Just let me know!



Data binding, decorators, css, rendering, loops, iteration

Sure! Here's a detailed explanation of the following **LWC (Lightning Web Components)** concepts with **examples**:

◆ 1. Data Binding in LWC

Data binding is how your HTML template stays in sync with your JavaScript code.

There are **two types**:

✓ a) One-way Data Binding (default)

- Template reflects value from JavaScript.
- Changes in JS update the HTML.

Example:

```
js

// greetingComponent.js
import { LightningElement } from 'lwc';

export default class GreetingComponent extends LightningElement {
  greeting = 'Hello';
}
```

```
html

<!-- greetingComponent.html -->
<template>
  <p>{greeting} Salesforce!</p> <!-- Output: Hello Salesforce! -->
</template>
```

✓ b) Two-way Data Binding (with input)

- Value updates both ways: from JS → HTML and vice versa.

Example:

```
js

// nameComponent.js
import { LightningElement } from 'lwc';

export default class NameComponent extends LightningElement {
  name = '';

  handleChange(event) {
    this.name = event.target.value;
  }
}
```

```
html

<!-- nameComponent.html -->
<template>
  <lightning-input label="Enter your name" value={name} onchange={handleChange}></lightning-input>
  <p>Your name is: {name}</p>
</template>
```

◆ 2. Decorators: @api, @track, @wire

✓ @api

- Marks a **public property/method**
- Can be accessed by parent components.

```
js

@api message = 'Hello';
```

✓ @track

- Makes **private object/array** properties reactive.
- Not needed for primitives.

```
js
@track contact = { name: 'Sam' };
```

Instead of **@track**, reassign the object to trigger reactivity:

```
js
this.contact = { ...this.contact, name: 'New Name' };
```

✓ @wire

- Used to get data from Apex or Salesforce data automatically.
- It makes the method/property reactive and automatically refreshed when dependencies change.

```
js
import { wire } from 'lwc';
import getAccounts from '@salesforce/apex/AccountController.getAccounts';

@wire(getAccounts) accounts;
```

◆ 3. CSS in LWC

- Each component has its own CSS file: **componentName.css**
- CSS is scoped to the component (encapsulated).

✓ Example:

myComponent.css

```
css
p {
  color: blue;
  font-size: 18px;
}
```

myComponent.html

```
html
<template>
  <p>This is styled!</p>
</template>
```

✓ Global Styles (shared CSS)

- Use **Lightning Design System (SLDS)** classes:

```
html

<div class="slds-box slds-theme_default">
  Styled using SLDS!
</div>
```

◆ 4. Rendering

Rendering refers to how the DOM is created from your component's template.

✓ Conditional Rendering: `if:true` and `if:false`

Example:

```
html

<template>
  <template if:true={showText}>
    <p>This text is visible</p>
  </template>
</template>
```

```
js

showText = true;
```

✓ Dynamic Template: `render()` method

Used to load different templates programmatically.

```
js

render() {
  return this.isLoggedIn ? templateLoggedIn : templateLoggedOut;
}
```

◆ 5. Loops and Iteration

LWC supports `for:each` directive for iterating over arrays.

✓ Basic Example:

```
js

students = ['Alice', 'Bob', 'Charlie'];
```

```
html

<template for:each={students} for:item="student">
  <p key={student}>{student}</p>
</template>
```

✓ Iteration of Objects:

```
js

contacts = [
  { id: 1, name: 'Sam', email: 'sam@test.com' },
  { id: 2, name: 'Alex', email: 'alex@test.com' }
];
```

```
html

<template for:each={contacts} for:item="contact">
  <div key={contact.id}>
    <p>Name: {contact.name}</p>
    <p>Email: {contact.email}</p>
  </div>
</template>
```

✓ Iteration with Index:

```
html

<template for:each={students} for:item="student" for:index="index">
  <p key={index}>#{index + 1} - {student}</p>
</template>
```

Summary Table

Concept	Purpose	Example Syntax
Data Binding	Sync JS data with UI	{name}
@api	Public property/method (parent → child)	@api name
@track	Reactive complex properties	@track user
@wire	Auto-fetch data from Apex or SFDC	@wire(...)
CSS	Style component locally or via SLDS	.my-class {}
Rendering	Show/hide or switch content dynamically	if:true, render()
Iteration	Loop over arrays/objects	for:each

If you'd like hands-on use cases like dynamic lists, conditional forms, etc., I can create those too. Let me know what area you want to focus on next!



I triduction to lwc, framework Aura vs lwc, structure, bundle, basic sfdx command

Absolutely—here's a clear, **structured explanation** for you:

🌟 Introduction to LWC (Lightning Web Components)

LWC (Lightning Web Components) is a **modern web framework** built by Salesforce for creating Lightning components.

✅ Key Points:

- Based on **modern web standards**: ECMAScript 6+, Web Components API.
- Faster performance and smaller footprint compared to Aura.
- More **standard JavaScript**, less proprietary abstraction.
- Components are **encapsulated**, modular, reusable.
- Runs natively in the browser.

⚡ Aura vs. LWC Framework

Here's a side-by-side comparison:

Feature	Aura Components	Lightning Web Components (LWC)
Standard	Proprietary Salesforce framework	Modern Web Standards (Custom Elements, Shadow DOM, Modules)
Performance	Heavier and more abstraction	Faster, lightweight
Learning Curve	Aura-specific syntax, more boilerplate	Familiar JavaScript & HTML
Data Binding	Two-way data binding	One-way by default
Component Encapsulation	Aura Locker Service	Native Shadow DOM encapsulation
Decorator Support	Not applicable	@api , @track , @wire decorators
Rendering Engine	Aura-specific rendering	Native browser rendering

Example of Aura vs LWC

Aura:

```
xml

<aura:component>
  <aura:attribute name="message" type="String" default="Hello Aura!" />
  <p>{!v.message}</p>
</aura:component>
```

LWC:

```
js

export default class HelloLwc extends LightningElement {
  message = 'Hello LWC!';
}
```

```
html

<template>
  <p>{message}</p>
</template>
```

LWC Structure & Bundle

An LWC component is a **bundle of files** stored in a single folder.

Example: helloWorld component:

```
cpp

helloWorld/
├── helloWorld.html      (template)
├── helloWorld.js        (controller logic)
├── helloWorld.js-meta.xml (metadata configuration)
├── helloWorld.css       (optional styles)
└── helloWorld.svg       (optional icon for App Builder)
```

✓ File Purpose:

- **.html:** Template markup (HTML with {} expressions)
- **.js:** JavaScript logic (properties, methods)
- **.js-meta.xml:** Configuration (visibility, targets)
- **.css:** Local styling
- **.svg:** Custom icon

Basic SFDX Commands

Salesforce CLI (sfdx) is the command-line tool to create and deploy LWCs.

1 Create a new project:

```
bash

sfdx force:project:create -n myLwcProject
```

2 Create an LWC component:

```
bash

sfdx force:LightningComponent:create \
  --type lwc \
```

```
--componentname helloWorld \  
--outputdir force-app/main/default/lwc
```

3 Authorize an org:

```
bash  
  
sfdx force:auth:web:login -a DevOrg
```

(-a assigns an alias)

4 Deploy to org:

```
bash  
  
sfdx force:source:deploy -p force-app/main/default/lwc/helloWorld
```

5 Retrieve from org:

```
bash  
  
sfdx force:source:retrieve -m LightningComponentBundle
```

6 Push to scratch org:

```
bash  
  
sfdx force:source:push
```

7 Open org in browser:

```
bash  
  
sfdx force:org:open
```

Quick Recap

✓ LWC:

- Built on **modern standards**
- Lightweight, faster than Aura
- Simple structure: HTML, JS, metadata
- Uses **@api**, **@track**, **@wire**

✓ Aura:

- Older framework
- More Salesforce-specific syntax
- Heavier abstraction

✓ SFDX:

- Create, deploy, retrieve components
- Automates development workflows

If you'd like, I can help you with **hands-on examples**, **deploying to Scratch Org**, or **writing your first LWC step by step**. Just let me know!



ECMA 6+, var vs let, functions, arrow functions, objects, constructor functions, es6 classes, call back functions in detail with multiple examples

Fantastic—this is a **core JavaScript (ECMAScript 6+)** topic that's critical both for LWC and modern web development.

I'll explain each concept clearly with **multiple examples and detailed commentary** so you get a solid understanding.

🌟 1 ECMAScript 6+ (ES6+) Overview

ECMAScript 6 (2015) introduced modern JavaScript features like:

- ✓ let and const
- ✓ Arrow functions
- ✓ Classes
- ✓ Template literals
- ✓ Destructuring
- ✓ Modules
- ✓ Promises

Everything since ES6 (ES7, ES8, etc.) builds on this foundation.

⚡ 2 var vs let vs const

Keyword	Scope	Hoisting	Reassignment
var	Function scope	Hoisted & initialized to undefined	Allowed
let	Block scope	Hoisted but not initialized (Temporal Dead Zone)	Allowed
const	Block scope	Hoisted but not initialized	No reassignment

✓ Example 1: Scope

```
javascript

function testVarLet() {
  if (true) {
    var a = 10;
    let b = 20;
  }
}
```

```
const c = 30;
console.log(a); // 10
console.log(b); // 20
console.log(c); // 30
}
console.log(a); // 10 (var is function-scoped)
// console.log(b); // Error: b is not defined
// console.log(c); // Error: c is not defined
}
testVarLet();
```

✓ Example 2: Hoisting

```
javascript

console.log(x); // undefined
var x = 5;

console.log(y); // ReferenceError
let y = 10;
```

✓ Example 3: Reassignment

```
javascript

const PI = 3.14;
// PI = 3.15; // Error: can't reassign const

let name = "Alice";
name = "Bob"; // OK

var age = 25;
age = 26; // OK
```

3 Functions

Normal (declarative) function:

```
javascript

function greet(name) {
    return "Hello, " + name;
}
console.log(greet("Sam")); // Hello, Sam
```

Function expression:

```
javascript

const add = function(a, b) {
    return a + b;
};
console.log(add(2, 3)); // 5
```

4 Arrow Functions

Syntax:

```
javascript
```

```
(param1, param2) => expression
```

✓ Example 1: Basic Arrow Function

```
javascript
```

```
const multiply = (x, y) => x * y;
console.log(multiply(3, 4)); // 12
```

✓ Example 2: With block body

```
javascript
```

```
const sayHello = (name) => {
  console.log("Hello, " + name);
};
sayHello("John");
```

✓ Example 3: Single parameter (no parentheses needed)

```
javascript
```

```
const square = n => n * n;
console.log(square(5)); // 25
```

✓ Note: Arrow functions DO NOT have their own this.

Example:

```
javascript
```

```
const obj = {
  value: 100,
  showValue: function() {
    console.log(this.value);
  },
  showValueArrow: () => {
    console.log(this.value);
  }
};

obj.showValue(); // 100
obj.showValueArrow(); // undefined (arrow doesn't bind `this`)
```

⚡ 5 Objects

Objects store key-value pairs.

✓ Example 1: Simple Object

```
javascript
```

```
const person = {
  name: "Alice",
  age: 30,
  greet() {
```

```

        console.log("Hello, I am " + this.name);
    }
};

console.log(person.name); // Alice
person.greet(); // Hello, I am Alice

```

✓ Example 2: Dynamic properties

```

javascript

const propName = "city";
const user = {
    [propName]: "New York"
};
console.log(user.city); // New York

```

✓ Example 3: Destructuring

```

javascript

const { name, age } = person;
console.log(name); // Alice
console.log(age); // 30

```

6 Constructor Functions

Before ES6 classes, we used constructor functions to create objects:

✓ Example:

```

javascript

function Car(make, model) {
    this.make = make;
    this.model = model;
}
Car.prototype.display = function() {
    console.log(this.make + " " + this.model);
};

const myCar = new Car("Toyota", "Corolla");
myCar.display(); // Toyota Corolla

```

7 ES6 Classes

Classes provide cleaner syntax for constructors + prototypes.

✓ Example 1: Basic Class

```

javascript

class Animal {
    constructor(name) {
        this.name = name;
    }
    speak() {
        console.log(this.name + " makes a sound.");
    }
}

```

```

    }
  }

  const dog = new Animal("Rex");
  dog.speak(); // Rex makes a sound.

```

✓ Example 2: Inheritance

```

javascript

class Dog extends Animal {
  speak() {
    console.log(this.name + " barks.");
  }
}

const d = new Dog("Buddy");
d.speak(); // Buddy barks.

```

Callback Functions

A **callback function** is passed to another function to be **called later**.

✓ Example 1: Simple callback

```

javascript

function processUserInput(callback) {
  const name = "Charlie";
  callback(name);
}

processUserInput(function(name) {
  console.log("Hello, " + name);
});
// Output: Hello, Charlie

```

✓ Example 2: Array iteration with callback

```

javascript

const numbers = [1, 2, 3];
numbers.forEach(function(num) {
  console.log(num * 2);
});
// Output: 2, 4, 6

```

✓ Example 3: Arrow function as callback

```

javascript

numbers.forEach(n => console.log(n * 3));

```

✓ Example 4: Callback for asynchronous operation

```

javascript

function fetchData(callback) {
  setTimeout(() => {

```

```
const data = "Some data";
callback(data);
}, 1000);
}

fetchData(result => {
  console.log("Received: " + result);
});
// After 1 second: Received: Some data
```

Quick Recap Table

Feature	Key Concept
var	Function-scoped, hoisted, reassignable
let	Block-scoped, TDZ, reassignable
const	Block-scoped, TDZ, no reassignment
Functions	Traditional declaration and expressions
Arrow Functions	Short syntax, no own this , concise
Objects	Key-value storage, methods, destructuring
Constructor Functions	Older way to create instances
ES6 Classes	Cleaner OOP, inheritance with extends
Callbacks	Functions passed to other functions, async handling

If you'd like, I can also show:

- **Promises and async/await**
- **Modules (import/export)**
- **Destructuring arrays/objects**
- **Rest/spread operators**
- **Map, filter, reduce**

Just let me know!



Promise, async/await, modules, import/export modules, destructuring, array methods, json explain in detail with multiple examples

Absolutely—let's cover these modern JavaScript topics step by step.
I'll **explain clearly with multiple examples** so you get a solid grasp.

Promises

Definition:

A **Promise** represents the *eventual completion* (or failure) of an asynchronous operation.

✓ Basic Structure:

```
javascript

const promise = new Promise((resolve, reject) => {
  // async work
  if (/* success */) {
    resolve(value);
  } else {
    reject(error);
  }
});
```

✓ Example 1: Simple Promise

```
javascript

const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded successfully!");
  }, 1000);
});

myPromise.then(result => {
  console.log(result);
});
// After 1 second: Data loaded successfully!
```

✓ Example 2: Handling errors

```
javascript

const errorPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    reject("Something went wrong!");
  }, 1000);
});

errorPromise
  .then(data => console.log("Data:", data))
  .catch(error => console.error("Error:", error));
// After 1 sec: Error: Something went wrong!
```

✓ Example 3: Chaining

```
javascript

const chainPromise = new Promise(resolve => {
  setTimeout(() => resolve(5), 500);
});

chainPromise
  .then(value => value * 2)
  .then(result => console.log(result)); // 10
```

★ 2 async / await

Definition:

- **async** makes a function return a Promise.
- **await** pauses execution until the Promise settles.

✓ Example 1: Basic async/await

```
javascript

async function fetchData() {
  return "Data fetched!";
}

fetchData().then(data => console.log(data)); // Data fetched!
```

✓ Example 2: Awaiting a Promise

```
javascript

function getNumber() {
  return new Promise(resolve => {
    setTimeout(() => resolve(42), 1000);
  });
}

async function showNumber() {
  const num = await getNumber();
  console.log(num);
}

showNumber(); // After 1 sec: 42
```

✓ Example 3: Error handling

```
javascript

function getError() {
  return new Promise( (_, reject) => {
    setTimeout(() => reject("Error occurred"), 500);
  });
}

async function handleError() {
  try {
    await getError();
  } catch (e) {
    console.error(e); // Error occurred
  }
}

handleError();
```

🌟 3 Modules

Purpose:

Split code into reusable files.

✓ Basic Syntax:

• Export:

```
javascript

export const PI = 3.1416;

export function square(x) {
  return x * x;
}

export default function greet(name) {
  return `Hello, ${name}`;
}
```

• Import:

```
javascript

import greet, { PI, square } from "./math.js";

console.log(PI); // 3.1416
console.log(square(4)); // 16
console.log(greet("Alice")); // Hello, Alice
```

✓ Note:

- **export default** allows importing without curly braces.
- Named exports must be imported using `{ }`.

★ 4 Destructuring

✓ Object Destructuring

```
javascript

const person = { name: "Bob", age: 40 };

const { name, age } = person;
console.log(name); // Bob
console.log(age); // 40
```

✓ Array Destructuring

```
javascript

const colors = ["red", "green", "blue"];
const [first, second] = colors;
console.log(first); // red
console.log(second); // green
```

✓ Default Values

```
javascript

const [x = 10, y = 20] = [5];
console.log(x); // 5
```

```
console.log(y); // 20
```

✓ Renaming

javascript

```
const user = { id: 1, username: "john" };  
const { id: userId, username: userName } = user;  
console.log(userId); // 1  
console.log(userName); // john
```

🌟 5 Array Methods

✓ forEach

javascript

```
[1, 2, 3].forEach(n => console.log(n * 2));  
// 2, 4, 6
```

✓ map

javascript

```
const nums = [1, 2, 3];  
const doubled = nums.map(n => n * 2);  
console.log(doubled); // [2, 4, 6]
```

✓ filter

javascript

```
const evens = nums.filter(n => n % 2 === 0);  
console.log(evens); // [2]
```

✓ reduce

javascript

```
const sum = nums.reduce((acc, curr) => acc + curr, 0);  
console.log(sum); // 6
```

✓ find

javascript

```
const item = nums.find(n => n > 1);  
console.log(item); // 2
```

✓ some

```
javascript

console.log(nums.some(n => n > 2)); // true
```

✓ every

```
javascript

console.log(nums.every(n => n < 4)); // true
```

✓ sort

```
javascript

const arr = [3, 1, 2];
arr.sort(); // [1, 2, 3]
```

🌟 6 JSON

✓ What is JSON?

- JavaScript Object Notation
- Text format for data exchange
- Keys must be strings

✓ Example:

```
json

{
  "name": "Alice",
  "age": 30,
  "skills": ["JS", "HTML"]
}
```

✓ Parse JSON to Object

```
javascript

const jsonString = '{"name":"Alice","age":30}';
const obj = JSON.parse(jsonString);
console.log(obj.name); // Alice
```

✓ Stringify Object to JSON

```
javascript

const data = { id: 1, product: "Book" };
const json = JSON.stringify(data);
console.log(json); // {"id":1,"product":"Book"}
```

Quick Recap Table

Feature	Purpose	Example
Promise	Handle async results	<code>new Promise((res, rej) => {})</code>
async/await	Syntactic sugar over Promises	<code>await myPromise()</code>
Modules	Split/reuse code	<code>import/export</code>
Destructuring	Extract values easily	<code>const {a} = obj</code>
Array Methods	Process collections concisely	<code>map, filter, reduce</code>
JSON	Exchange data between systems	<code>JSON.parse, JSON.stringify</code>

✅ If you'd like, I can also explain:

- **Rest and spread operators**
- **Optional chaining**
- **Template literals**
- **Closures**
- **Generators**

Just let me know!